Вопросы к зачёту за I семестр по программированию на C++ (Вопросы 1–8)

Полный конспект со всеми подпунктами программы: дополнительный код, IEEE 754, intX_t, указатели и ссылки, сдвиги и parity за O(log n), 4 этапа трансляции, 5 сегментов памяти (.text, .data, .bss, heap, stack), сдвиг через 3 reverse, swap через +/XOR и все виды сортировок.

Оглавление

1. **Вопрос №1.** Переменные и типы данных в C++. Область видимости переменной.
2. **Вопрос №2.** Условные ветвления и циклы. Операторы.
3. **Вопрос №3.** Составные типы данных - указатель и ссылка. Операторы new / delete.
4. **Вопрос №4.** Функции, перегрузки, значения параметров по умолчанию. Рекурсия.
5. **Вопрос №5.** Побитовые операторы, сдвиги, основные приёмы.
6. **Вопрос №6.** Процесс трансляции C++ кода в исполняемый файл. Области памяти исполняемой программы на C++.
7. **Вопрос №7.** Алгоритмы. Пос-ть Фиббоначи. Линейный и бинарный поиск. Reverse, shift массива. Соответсвующие оценки сложности. Swap целых чисел через + и xor. Вычисление бита чётности (parity bit) за n и log(n).
8. **Вопрос №8.** Алгоритмы. Получение случайного числа в заданном диапазоне при помощи ф-ции rand(), принцип её работы. Сортировки n^2: выбором, вставкой, пузырьком (оптимизации: перемешиванием, расчёской). Сортировки n^log(n): слиянием, быстрая, пирамидальная. Сортировки n: подсчётом.

ВОПРОС №1 ИЗ 8 · ТИПЫ ДАННЫХ И ПАМЯТЬ

1. Переменные и типы данных в C++. Область видимости переменной.

Фундаментальный разбор синтаксиса переменных, базовых примитивных типов C++, двоичного дополнительного кода (Two’s Complement), стандарта IEEE 754

(float, double, long double), псевдонимов typedef/using, фиксированных типов intX_t/uintX_t, систем счисления bin/hex, квалификаторов const/constexpr и областей видимости.

Ключевые тезисы:

- Дополнительный код (Two’s Complement) для отрицательного числа -X получается инверсией всех битов числа X и прибавлением 1: (-X) = (~X) + 1. Диапазон знакового N-битного типа: [-2^(N-1) .. 2^(N-1) - 1], беззнакового: [0 .. 2^N - 1].
- Вещественные числа хранятся по стандарту IEEE 754: Знак (1 бит) + Смещённая экспонента + Мантисса (дробная часть с неявной ведущей единицей 1.M).
- int8_t, int16_t, int32_t, int64_t из <cstdint> — это не новые типы, а typedef-псевдонимы стандартных типов с гарантированным размером в битах.
- const означает «неизменяемый после инициализации» (может вычисляться в рантайме), а constexpr гарантирует вычисление значения на этапе компиляции.

1. Синтаксис объявления, инициализации, присваивания и базовые типы данных

Переменная — это именованная область оперативной памяти заданного типа. Жизненный цикл работы с переменной включает: 1) Объявление и определение: Тип имя; (например, int count; — выделяет память, но в локальной области оставляет «мусор»); 2) Инициализацию (задание начального значения в момент создания): копирующую int a = 5;, прямую int a(5); или списочную (uniform initialization C++11, защищающую от сужения типов) int a{5};; 3) Присваивание: запись нового значения в уже существующую переменную: a = 10;.

Четыре группы базовых (примитивных / фундаментальных) типов данных в C++:

Группа типов	**Ключевые слова C++**	**Типичный размер (x86_64)**	**Особенности представления**
Целочисленные (Integral)	short, int, long, long long (co	2, 4, 4/8, 8 байт (16, 32,	Хранятся точно. Отрицательные числа

Группа типов	Ключевые слова C++	Типичный размер (x86_64)	Особенности представления
	знаком signed и без знака unsigned)	64 бита)	кодируются в дополнительном коде (Two’s Complement).
Символьные (Character)	char, signed char, unsigned char, wchar_t, char16_t, char32_t	char — строго 1 байт (8 бит)	Хранят числовой код символа (ASCII для char) и одновременно являются малоразрядными целыми числами.
Логический (Boolean)	bool	1 байт (8 бит)	Принимает значения false (0) или true (1). Любое ненулевое число при приведении к bool даёт true (1).
Вещественные (Floating-point)	float, double, long double	4 байта, 8 байт, 8/10/16 байт	Числа с плавающей точкой по стандарту IEEE 754 (приближённое хранение с фиксированной точностью мантиссы).

2. Устройство чисел со знаком (Дополнительный код), bin/hex и расчёт диапазонов

Двоичная (bin, префикс 0b в C++14: 0b1010 = 10) и шестнадцатеричная (hex, префикс 0x: 0xFF = 255) системы счисления тесно связаны: 1 шестнадцатеричная цифра (тетрада / ниббл) кодирует ровно 4 бита (от 0000 = 0x0 до 1111 = 0xF), поэтому 1 байт (8 бит) всегда записывается ровно двумя hex-цифрами (от 0x00 до 0xFF).

Количество различных комбинаций из N бит равно 2^N. Для беззнакового типа (unsigned) все N бит идут на величину числа, поэтому min = 0, а max = 2^N - 1 (все единицы в двоичной записи, например для 8 бит: 0b11111111 = 2^8 - 1 = 255).

Как устроены целые числа со знаком (Дополнительный код — Two’s Complement)? Старший (самый левый, N-1)-й бит отводится под знак: 0 — число неотрицательное (>= 0), 1 — число отрицательное (< 0). Чтобы получить код отрицательного числа -X из положительного X, нужно: 1) инвертировать все биты числа X (~X — обратный код), 2) прибавить к результату единицу (+ 1). Например, для 8-битного типа: +5 =

00000101; инвертируем биты: 11111010; прибавляем 1: -5 = 11111011. Преимущество дополнительного кода: процессор складывает положительные и отрицательные числа одним и тем же сумматором без отдельных схем вычитания, а нуль имеет единственное представление (00000000).

Диапазон знакового N-битного типа: от -2^(N-1) (код 100...00, например -128 для 8 бит) до +2^(N-1) - 1 (код 011...11, например +127 для 8 бит). Одно место в положительной половине занимает число 0, поэтому модуль минимального отрицательного числа на 1 больше максимального положительного!

3. Устройство вещественных чисел с плавающей точкой (float, double, long double)

По стандарту IEEE 754 любое ненулевое вещественное число представляется в экспоненциальной двоичной нормализованной форме: V = (-1)^S * (1.M)_2 * 2^(E - Bias), где битовое поле делится на 3 части: 1) Знак S (1 старший бит: 0 для +, 1 для -); 2) Смещённый порядок (экспонента E, хранится как целое число со смещением Bias, чтобы экспонента могла быть отрицательной без знакового бита); 3) Мантисса M (дробная часть после запятой; целая единица «1.» перед точкой подразумевается неявно и не тратит бит памяти!).

Тип C++	Всего бит (байт)	Разбиение битов (S / E / M)	Десятичная точность и диапазон
float (одинарная точность)	32 бита (4 байта)	1 бит знака + 8 бит экспоненты (Bias=127) + 23 бита мантиссы	6–7 значащих десятичных цифр (~1.18e-38 .. ~3.4e+38)
double (двойная точность)	64 бита (8 байт)	1 бит знака + 11 бит экспоненты (Bias=1023) + 52 бита мантиссы	15–16 значащих десятичных цифр (~2.23e-308 .. ~1.8e+308)
long double (расширенная точность)	80 бит (10/16 байт в GCC x86) или 64 бита (MSVC)	1 бит знака + 15 бит экспоненты + 64 бита мантиссы (x87)	18–19 значащих десятичных цифр (на платформах с 80/128-битным FPU)

4. Оператор typedef, типы intX_t / uintX_t, const vs constexpr и область видимости

Ключевое слово typedef (и современная форма C++11 using) НЕ создаёт новый тип данных, а лишь объявляет синоним (псевдоним / alias) для уже существующего типа: typedef unsigned long long ull; (или using ull = unsigned long long;).

Что такое int8_t, uint8_t, int16_t, int32_t, int64_t (заголовок <cstdint>)? Стандарт C++ гарантирует для обычных типов лишь минимальные размеры (например, int обязан быть >= 16 бит, поэтому на микроконтроллере AVR int занимает 2 байта, а на ПК — 4 байта). Чтобы код был переносимым и имел фиксированный размер на любой платформе, в <cstdint> через typedef определены псевдонимы: int8_t (signed char), uint8_t (unsigned char), int16_t (short), int32_t (int), int64_t (long long). Важная тонкость: так как int8_t/uint8_t — это typedef для char, поток std::cout печатает их как ASCII-символ, а не число (для вывода числом нужно привести к +x или static_cast<int>(x))!

- Разница между const и constexpr: 1) const («обещание не менять») означает, что переменную нельзя модифицировать после инициализации, но её значение может вычисляться во время работы программы (например, const int x = rand(); или ввод с клавиатуры). 2) constexpr (C++11) требует, чтобы значение переменной или функции было гарантированно вычислено НА ЭТАПЕ КОМПИЛЯЦИИ (Compile-time constant) и могло использоваться как размер статического массива или метка case.
- Область видимости переменной (Scope): 1) Глобальная область видимости (Global / File / Namespace scope) — переменная объявлена вне всех функций и классов; видна от места объявления до конца файла (если затенена локальной переменной с тем же именем, доступна через унарный оператор ::var). 2) Локальная область видимости (Local / Block scope) — переменная объявлена внутри функции, параметров функции или любого блока фигурных скобок { ... } (включая блоки if, for, while); видна только от точки объявления до закрывающей скобки } своего блока.

Пример кода C++: Диапазоны типов, дополнительный код, intX_t, const vs constexpr и области видимости

Демонстрация работы с системами счисления bin/hex, проверки дополнительного кода (-x == ~x + 1), фиксированных типов <cstdint> и разрешения затенения имён через ::.

```
#include <iostream>
#include <cstdint>
#include <limits>
```

```
int globalValue = 777; // Глобальная область видимости

constexpr int compileTimeCube(int n) {
    return n * n * n; // Вычисляется во время компиляции
}

int main() {
    // 1. Системы счисления bin (0b) и hex (0x) и дополнительный код
    int8_t pos = 0b00000101; // 5 в двоичной системе
    int8_t neg = static_cast<int8_t>(~pos + 1); // Дополнительный код: -5
(0xFB)
    uint8_t maxU8 = 0xFF;    // 255 в шестнадцатеричной системе (2^8 - 1)

    std::cout << "pos = " << static_cast<int>(pos)
              << ", neg (~pos + 1) = " << static_cast<int>(neg)
              << ", maxU8 = " << static_cast<int>(maxU8) << "
";

    std::cout << "Диапазон int16_t: [" <<
std::numeric_limits<int16_t>::min()
              << " .. " << std::numeric_limits<int16_t>::max() << "]
";

    // 2. Разница const и constexpr
    constexpr int arrSize = compileTimeCube(2); // 8 — известно при
компиляции!
    int runtimeInput = 42;
    const int readOnlyRuntime = runtimeInput * 2; // Инициализируется в
рантайме
    int staticArr[arrSize] = {readOnlyRuntime};

    // 3. Локальная и блочная область видимости (затенение имён)
    int globalValue = 100; // Локальная переменная в main затеняет
глобальную
    {
        int globalValue = 10; // Переменная внутри вложенного блока {}
        std::cout << "Внутри блока {}: local = " << globalValue
                  << ", глобальная ::globalValue = " << ::globalValue <<
"
";
    }
    std::cout << "В функции main: local = " << globalValue
              << ", staticArr[0] = " << staticArr[0] << "
";
    return 0;
}
```

Результат выполнения (stdout):

```
pos = 5, neg (~pos + 1) = -5, maxU8 = 255
Диапазон int16_t: [-32768 .. 32767]
Внутри блока {}: local = 10, глобальная ::globalValue = 777
В функции main: local = 100, staticArr[0] = 84
```

Тонкость / на что обратить внимание: На зачёте часто спрашивают: чему равен результат (uint8_t)0 - 1? Ответ: из-за модульной арифметики беззнаковых чисел по модулю 2^8 получится 255 (все 8 бит станут равны 1: 0xFF)! А почему 0.1 + 0.2 != 0.3 в double? Потому что дробь 1/10 в двоичной системе является бесконечной периодической дробью 0.0(0011)_2 и округляется при хранении в 52 битах мантиссы.

ВОПРОС №2 ИЗ 8 · ТИПЫ ДАННЫХ И ПАМЯТЬ

2. Условные ветвления и циклы. Операторы.

Полный разбор конструкций ветвления (if / else if / else, switch и допустимых типов его селектора), циклов (while, do-while, 3 сегмента for и точный порядок их выполнения), переходов (break, continue, goto) и операторов C++ как функций над операндами.

Ключевые тезисы:

- switch работает ТОЛЬКО с целочисленными типами (int, char, short, long, long long) и перечислениями (enum / enum class). Со строками std::string и вещественными double он НЕ работает!
- Порядок выполнения for (A; B; C) { Body; }: 1) один раз выполняется блок инициализации A; 2) проверяется условие B (если false — выход); 3) выполняется тело цикла Body; 4) выполняется шаг C; 5) переход к пункту 2.
- Префиксный ++x сначала изменяет переменную и возвращает ссылку на неё (l-value, без создания временной копии); постфиксный x++ создаёт временную копию старого значения (r-value), увеличивает переменную и возвращает старую копию.

1. Условные ветвления: if / else if / else и switch-case

Конструкции if (условие) и цепочки if (...) { ... } else if (...) { ... } else { ... } позволяют направить поток выполнения по одной из альтернативных веток. Любое арифметическое выражение или указатель в условии if неявно приводится к bool (0 или nullptr → false, любое другое значение → true).

Варианты синтаксиса фигурных скобок: если тело ветки if, else или цикла состоит ровно из ОДНОЙ инструкции, фигурные скобки { } синтаксически можно опустить. Однако если внутри написать две инструкции без скобок, вторая инструкция выполнится безусловно! Кроме того, при вложенных if без скобок правило C++ гласит: else всегда привязывается к БЛИЖАЙШЕМУ вышестоящему незакрытому if (проблема «висячего else» — dangling else).

Конструкция множественного выбора switch (селектор) сопоставляет значение селектора с константными метками case CONST_VAL: (значения в case обязаны быть известны на этапе компиляции — литералы или constexpr):

- С какими типами работает switch? Исключительно с целочисленными типами (char, short, int, long, long long и их unsigned-вариантами), типом bool и перечислимыми типами (enum / enum class). Использовать float, double, указатели или std::string в switch запрещено стандартом языка!
- Почему нужен break в switch? Метки case — это просто точки входа (как метки для goto). После совпадения с case процессор выполняет все инструкции подряд вниз до конца блока switch или до первого оператора break;. Если break опущен, происходит «проваливание» (fallthrough) в код следующего case.

2. Циклы while, do-while, for и операторы управления break, continue, goto

1) while (условие) { тело } — цикл с предусловием (проверяет условие ДО каждой итерации; может не выполниться ни разу). 2) do { тело } while (условие); — цикл с постусловием (сначала выполняет тело, затем проверяет условие; гарантированно выполняется минимум 1 раз; завершается точкой с запятой!).

3) Цикл for (Сегмент_1_Инициализация; Сегмент_2_Условие; Сегмент_3_Итерация) { Тело_Цикла } делится на 3 сегмента в заголовке, разделённых точкой с запятой (;). Любой из трёх сегментов (или все три: for (;;)) может быть пустым.

- break — немедленно прерывает самый внутренний объемлющий цикл (for, while, do-while) или конструкцию switch.

- continue — прерывает только ТЕКУЩУЮ итерацию цикла: пропускает оставшуюся часть тела цикла и переходит к проверке условия (в while/do-while) или к Сегменту 3 инкремента (в цикле for).
- goto метка; — безусловный переход к строке с меткой метка: в пределах той же функции. На практике допустим только для мгновенного выхода сразу из нескольких глубоко вложенных циклов.

Шаг исполнения цикла for	**Какой сегмент выполняется**	**Описание действия**
Шаг 1 (при входе в цикл)	Сегмент 1: Инициализация (int i = 0)	Выполняется РОВНО ОДИН РАЗ перед началом первой итерации.
Шаг 2 (перед каждой итерацией)	Сегмент 2: Условие (i < n)	Проверяется перед каждым входом в тело. Если true (или пусто) — идём в Шаг 3; если false — цикл завершается.
Шаг 3 (основное действие)	Тело цикла { ... }	Выполняются инструкции внутри цикла.
Шаг 4 (после тела цикла)	Сегмент 3: Модификация (++i)	Выполняется ПОСЛЕ завершения тела цикла (и после срабатывания continue!), затем управление возвращается на Шаг 2.

3. Оператор как функция: арность, инкремент/декремент и операторы присваивания

В C++ любой оператор можно рассматривать как функцию с особым синтаксисом вызова: операнды выступают в роли входных аргументов функции, а результат вычисления выражения — это возвращаемое значение функции (которое имеет свой тип и категорию значения: l-value или r-value). В объектно-ориентированном C++ большинство операторов буквально записываются как функции: operator+(a, b), operator=(a, b).

По количеству принимаемых операндов (арности) операторы делятся на: 1) Унарные (1 операнд): префиксные (+x, -x, ++x, --x, !x, ~x, *ptr, &var, sizeof(x)) и постфиксные (x++, x--); 2) Бинарные (2 операнда): арифметические (+, -, *, /, %), сравнения (==, !=, <, >), логические (&&, ||), побитовые (&, |, ^, <<, >>), присваивания (=, +=, -=); 3) Тернарный (3 операнда — единственный в C++): условный оператор cond ? exprTrue : exprFalse.

Разница между префиксным (++x / --x) и постфиксным (x++ / x--) инкрементом и декрементом: Префиксный оператор ++x сначала увеличивает саму переменную в памяти на 1, а затем возвращает ссылку на эту же изменённую переменную (l-value). Постфиксный оператор x++ обязан сначала создать временную копию старого значения переменной, затем увеличить саму переменную на 1 и вернуть созданную временную копию со старым значением (r-value). Поэтому в циклах for всегда предпочтительнее писать ++i, чтобы не создавать лишнюю временную копию на каждой итерации.

Оператор присваивания (=) и составные операторы присваивания (+=, -=, *=, /=, %=, &=, |=, ^=, <<=, >>=) имеют ПРАВУЮ ассоциативность (выполняются справа налево) и возвращают ссылку (l-value) на левый операнд, что позволяет писать a = b = c = 5;. При этом составной оператор a += expr эффективнее, чем a = a + expr, так как адрес левого операнда a вычисляется ровно один раз.

Пример кода C++: Порядок сегментов for, ветвление switch, префиксный/постфиксный инкремент и тернарный оператор

Наглядная трассировка работы циклов с continue/break, составных операторов присваивания и различия ++x и x++.

```
#include <iostream>

int main() {
    // 1. Демонстрация префиксного (++x) и постфиксного (x++) инкремента
    int x = 10;
    int postResult = x++; // postResult получит старое значение 10, а x
станет 11
    int prefResult = ++x; // x сначала станет 12, и prefResult получит 12
    std::cout << "postResult = " << postResult << ", prefResult = " <<
prefResult
              << ", итоговый x = " << x << "
";

    // 2. Цикл for с continue и break + составное присваивание +=
    int sumOdd = 0;
    for (int i = 1; i <= 10; ++i) { // Сегмент 1 -> Сегмент 2 -> Тело ->
Сегмент 3
        if (i % 2 == 0) continue;   // Пропускаем чётные числа (прыжок к
++i)
        if (i > 7) break;           // Выходим из цикла при i = 9
        sumOdd += i;                // Составной оператор присваивания (1
+ 3 + 5 + 7)
    }
    std::cout << "Сумма нечётных (1..7): " << sumOdd << "
```

```
";

    // 3. Тернарный оператор (?:) и switch по целочисленному коду
операции
    char op = '*';
    int a = 6, b = 7;
    int res = 0;
    switch (op) {
        case '+': res = a + b; break;
        case '*': res = a * b; break;
        default:  res = 0;     break;
    }
    std::cout << "Результат switch (" << op << "): " << res
              << " (" << ((res > 20) ? "Больше 20" : "Не больше 20") <<
")
";
    return 0;
}
```

Результат выполнения (stdout):

```
postResult = 10, prefResult = 12, итоговый x = 12
Сумма нечётных (1..7): 16
Результат switch (*): 42 (Больше 20)
```

Тонкость / на что обратить внимание: Классический вопрос на зачёте: что произойдёт, если внутри цикла while (i < 10) вызвать continue;, когда инкремент ++i находится в самом конце тела цикла после continue? Цикл станет бесконечным! А в цикле for (int i = 0; i < 10; ++i) оператор continue переходит прямо к сегменту ++i, и зацикливания не происходит.

ВОПРОС №3 ИЗ 8 · УКАЗАТЕЛИ, ССЫЛКИ И ПАМЯТЬ

3. Составные типы данных - указатель и ссылка. Операторы new / delete.

Глубокий разбор указателей и ссылок в C++: чтение объявлений с const справа налево, операторы * и &, адресная арифметика и эквивалентность a[i] == *(a + i), динамическое выделение и очистка памяти new/delete для 2D-матрицы (int**), типичные ошибки работы с памятью и фундаментальные отличия ссылки от указателя.

Ключевые тезисы:

- Чтение объявлений с const справа налево: int* const p (константный указатель на int), const int* p или int const* p (указатель на константный int), const int* const p (константный указатель на константный int).
- Оператор индексации arr[i] — это синтаксический сахар для адресной арифметики *(arr + i), где сдвиг указателя на +i автоматически умножается на sizeof(T) байт.
- Ссылка (T& ref = x) — это псевдоним (альяс) существующего объекта: она обязана быть инициализирована при создании, не может быть nullptr, её нельзя перепривязать к другому объекту и над ней нет адресной арифметики.

1. Указатели: синтаксис, операторы & и *, правило чтения const справа налево

Указатель (Pointer) — это переменная, которая хранит в качестве своего значения адрес другой ячейки оперативной памяти (на 64-битной платформе любой указатель занимает ровно 8 байт; на 32-битной — 4 байта). Синтаксис объявления: Тип* имя_указателя = nullptr;. Тип указателя сообщает компилятору, на объект какого типа (и какого размера в байтах) указывает данный адрес.

Два базовых унарных оператора для работы с указателями: 1) Оператор взятия адреса (&var) — принимает l-value переменную и возвращает её адрес в памяти (тип T*); 2) Оператор разыменования (*ptr) — обращается по адресу, сохранённому в указателе ptr, и возвращает сам объект (l-value типа T), лежащий в этой ячейке памяти.

Правило чтения const при объявлении указателей (чтение справа налево, читая символ * как «указатель на»):

Объявление C++	**Чтение справа налево**	**Можно менять адрес (p = &y)?**	**Можно менять значение (*p = 5)?**
int * p;	p — это указатель (*) на целое число (int).	ДА	ДА
const int * p; (или int const *	p — это указатель (*) на константное целое (int	ДА (можно перенаправить на	НЕТ (ошибка компиляции при

Объявление C++	Чтение справа налево	Можно менять адрес (p = &y)?	Можно менять значение (*p = 5)?
p;)	const) [указатель на константу].	другой адрес)	*p = 5)
int * const p = &x;	p — это константный (const) указатель (*) на целое число (int) [константный указатель].	НЕТ (привязан к адресу &x навсегда)	ДА (значение переменной x менять можно)
const int * const p = &x;	p — это константный (const) указатель (*) на константное целое (const int).	НЕТ	НЕТ

2. Арифметика указателей, оператор [] и динамическая память new / delete

Прибавление или вычитание целого числа k к указателю (ptr + k, ptr - k, ++ptr, --ptr) сдвигает адрес в памяти не на k байт, а на k * sizeof(T) байт (ровно на k элементов типа T). Разность двух указателей одного типа (p2 - p1) возвращает количество элементов типа T между ними. Оператор квадратные скобки ptr[i] по стандарту C++ раскрывается компилятором как *(ptr + i).

Операторы динамического управления памятью в Куче (Heap): new Тип выделяет память под один объект и вызывает его конструктор (освобождается оператором delete ptr;); оператор new Тип[N] выделяет непрерывный массив из N элементов в куче (освобождается строго парным оператором delete[] ptr;).

Проблемы и ошибки при работе с указателями:

- 1. Wild pointer (Garbage value / «дикий» неинициализированный указатель): int* p; *p = 10; — указатель хранит случайный мусор со стека вместо валидного адреса. Разыменование ведёт к Segmentation Fault / Access Violation. Решение: всегда инициализировать указатель nullptr.
- 2. Memory leak (Утечка памяти): память выделена через new / new[], но указатель на неё перезаписан другим адресом или уничтожен при выходе из функции без вызова delete / delete[]. Выделенная память в куче остаётся заблокированной до конца работы программы.

- 3. Ownership problem (Проблема владения ресурсом) и Dangling pointer (Висячий указатель): когда несколько указателей в разных частях программы ссылаются на один и тот же динамический объект, возникает неоднозначность: кто именно обязан вызвать delete? Это приводит либо к двойному удалению (Double Free — крах программы), либо к обращению через второй указатель после того, как первый уже вызвал delete (Use-after-free / Dangling pointer). В современном C++ проблема владения решается умными указателями (std::unique_ptr, std::shared_ptr).

3. Ссылки (References) и их отличия от указателей

Ссылка (объявляется как Тип& имя_ссылки = существующая_переменная;) — это псевдоним (альтернативное имя / alias) для уже существующего в памяти объекта. Любая операция над ссылкой является прямой операцией над тем объектом, к которому она привязана. На уровне машинного кода компилятор либо вообще не тратит память на ссылку (подставляя адрес переменной напрямую), либо внутри функций реализует её скрытым константным указателем int* const.

Критерий сравнения	**Указатель (T* ptr)**	**Ссылка (T& ref)**
Инициализация при объявлении	Необязательна (но рекомендуется = nullptr).	СТРОГО ОБЯЗАТЕЛЬНА (int& r; — ошибка компиляции).
Может ли быть пустым (nullptr)?	Да, может хранить nullptr (ни на что не указывать).	НЕТ, ссылка всегда обязана ссылаться на существующий объект.
Перепривязка к другому объекту	Можно в любой момент перенаправить (ptr = &otherVar;).	НЕЛЬЗЯ перепривязать! Запись ref = otherVar просто скопирует значение otherVar в исходный объект.
Память и адрес (&)	Имеет собственный адрес в памяти (&ptr) и занимает 8 байт (x64).	Оператор &ref возвращает адрес самого исходного объекта, а sizeof(ref) равен sizeof(T).
Адресная арифметика	Поддерживается (++ptr, ptr + i, ptr[i]).	Арифметика ссылок НЕ существует.

Пример кода C++: Выделение и очистка динамической памяти под 2D-матрицу (int) и сравнение указателя со ссылкой**

Эталонный код создания двумерного динамического массива как указателя на массив указателей и его пошагового освобождения без утечек памяти.

```
#include <iostream>

int main() {
    // 1. Динамическое выделение памяти под матрицу (rows x cols) через
int**
    int rows = 3, cols = 4;
    int** matrix = new int*[rows]; // Выделяем массив указателей на
строки
    for (int i = 0; i < rows; ++i) {
        matrix[i] = new int[cols]; // Выделяем память под каждую
отдельную строку
        for (int j = 0; j < cols; ++j) {
            // Заполняем через адресную арифметику (эквивалентно
matrix[i][j])
            *(*(matrix + i) + j) = (i + 1) * 10 + j;
        }
    }

    std::cout << "Элемент matrix[1][2] = " << matrix[1][2] << "
";

    // 2. Правильная очистка динамической памяти под матрицу (в обратном
порядке!)
    for (int i = 0; i < rows; ++i) {
        delete[] matrix[i]; // Сначала освобождаем каждую строку
    }
    delete[] matrix;        // Затем освобождаем массив указателей
    matrix = nullptr;       // Защита от висячего указателя (dangling
pointer)

    // 3. Ссылка как псевдоним переменной vs Указатель
    int original = 50;
    int& ref = original;    // ref — второе имя для original
    int* ptr = &original;   // ptr хранит адрес original

    ref += 25;              // Меняет original на 75
    *ptr += 25;             // Меняет original на 100
    std::cout << "original = " << original << ", &ref == ptr: "
              << std::boolalpha << (&ref == ptr) << "
";
    return 0;
}
```

Результат выполнения (stdout):

```
Элемент matrix[1][2] = 22
```

```
original = 100, &ref == ptr: true
```

Тонкость / на что обратить внимание: Если на зачёте попросят прочитать объявление const int * const * p справа налево: «p — это указатель (*) на константный указатель (* const) на константное целое число (const int)».

ВОПРОС №4 ИЗ 8 · ФУНКЦИИ И АЛГОРИТМЫ

4. Функции, перегрузки, значения параметров по умолчанию. Рекурсия.

Разбор устройства функций в C++: сигнатура и тело, передача аргументов по значению и по ссылке, синтаксис указателя на функцию, правила перегрузки функций, требование размещения параметров по умолчанию в конце списка, а также механизм рекурсии и причина переполнения стека (Stack Overflow).

Ключевые тезисы:

- В сигнатуру функции для разрешения перегрузки входят имя функции и упорядоченный список типов её параметров (тип возвращаемого значения в сигнатуру перегрузки НЕ входит!).
- Параметры со значениями по умолчанию обязаны располагаться непрерывно в самом КОНЦЕ (справа) списка параметров.
- Передача по значению (int x) копирует аргумент в стек, а передача по ссылке (int& x или const int& x) работает напрямую с внешней переменной без копирования.
- Рекурсивная функция обязана иметь базовый случай (терминальное условие выхода); иначе бесконечные вызовы исчерпают стек и вызовут Stack Overflow.

1. Объявление, сигнатура, тело функции и передача по значению vs по ссылке

Функция в C++ состоит из заголовка: ТипВозврата имяФункции(СписокФормальныхПараметров) и тела функции { инструкции; return

результат; }. Объявление (прототип) функции содержит только заголовок с точкой с запятой и сообщает компилятору типы для проверки вызовов, а определение содержит тело функции.

Сигнатура функции (Function Signature) — это совокупность имени функции и типов её формальных параметров (включая квалификаторы const/volatile у ссылок и указателей). Важно: тип возвращаемого значения НЕ является частью сигнатуры при разрешении перегрузки!

Разница передачи параметров по значению и по ссылке: 1) При передаче по значению (void f(int val)) создаётся независимая локальная копия аргумента в стековом кадре функции. Изменение val внутри функции никак не меняет переменную в вызывающем коде, но для крупных структур и std::vector копирование обходится дорого. 2) При передаче по ссылке (void f(int& ref)) копия не создаётся — функция получает прямой доступ к исходной переменной и может её модифицировать. Если нужно передать большой объект быстро без копирования, но запретить его менять, используют константную ссылку: void f(const std::string& text).

2. Перегрузка функций, параметры по умолчанию и указатель на функцию

Перегрузка функций (Function Overloading) позволяет объявлять несколько функций с одним и тем же именем в одной области видимости при условии, что они отличаются количеством параметров, типами параметров или порядком их следования. Компилятор различает их за счёт декорирования имён (Name Mangling).

Значения параметров по умолчанию (Default arguments): позволяют задать в объявлении функции значение параметра, которое подставится автоматически, если вызывающий код опустил этот аргумент: double logVal(double x, double base = 2.0);. Жёсткое правило C++: все параметры со значениями по умолчанию ОБЯЗАНЫ находиться в самом конце (справа) списка параметров! Почему? Потому что аргументы при вызове сопоставляются с параметрами позиционно слева направо — если бы параметр по умолчанию стоял посередине (void f(int a = 1, int b)), при вызове f(5) компилятор не смог бы понять, передана ли пятёрка в a или в b.

Синтаксис объявления указателя на функцию: ТипВозврата (*имяУказателя) (ТипыПараметров); (например: int (*opPtr)(int, int) = &add;). Круглые скобки вокруг (*имяУказателя) обязательны, иначе получится прототип функции, возвращающей указатель!

3. Рекурсия и проблема переполнения стека (Stack Overflow)

Рекурсивная функция — это функция, которая прямо или косвенно вызывает саму себя. Любая корректная рекурсия строится из двух частей: 1) Базовый случай (терминальное условие выхода), где ответ возвращается сразу без рекурсивного вызова (например, if (n <= 1) return 1;); 2) Рекурсивный шаг, сводящий задачу к подзадаче меньшего размера.

Почему возникает проблема переполнения стека (Stack Overflow)? При КАЖДОМ вызове функции процессор выделяет в стеке вызовов (Call Stack, размер которого в ОС обычно ограничен 1–8 МБ) новый стековый кадр (Stack Frame), куда сохраняет адрес возврата, значения параметров и локальные переменные. Если в рекурсивной функции забыть базовый случай, или шаг рекурсии не приближает аргумент к базовому случаю, или глубина рекурсии слишком велика (например, 1 000 000 вложенных вызовов), свободное место в сегменте стека заканчивается, указатель стека SP выходит за границу выделенной страницы памяти, и операционная система аварийно убивает процесс с ошибкой Stack Overflow (Segmentation Fault).

Пример кода C++: Перегрузка функций, параметры по умолчанию, передача по ссылке, указатель на функцию и рекурсия

Демонстрирует все концепции билета №4 в едином рабочем примере.

```
#include <iostream>

// 1. Передача по значению vs Передача по ссылке
void modifyValue(int val, int& ref) {
    val += 100; // Меняет только локальную копию
    ref += 100; // Меняет исходную переменную вызывающего кода
}

// 2. Перегрузка функций + параметр по умолчанию в конце списка
int compute(int a, int b, int multiplier = 1) {
    return (a + b) * multiplier;
}

double compute(double a, double b) {
    return a * b;
}

// 3. Рекурсивная функция вычисления факториала с базовым случаем
unsigned long long factorial(unsigned int n) {
    if (n <= 1) return 1ULL;        // Базовый случай (защита от
бесконечной рекурсии)
    return n * factorial(n - 1);    // Рекурсивный шаг
```

```
}

int main() {
    int x = 5, y = 5;
    modifyValue(x, y);
    std::cout << "После modifyValue: x (по значению) = " << x
              << ", y (по ссылке) = " << y << "
";

    std::cout << "compute(3, 4) [по умолчанию mult=1] = " << compute(3,
4) << "
";
    std::cout << "compute(3, 4, 10) [явно mult=10] = " << compute(3, 4,
10) << "
";
    std::cout << "compute(2.5, 4.0) [перегрузка double] = " <<
compute(2.5, 4.0) << "
";

    // 4. Указатель на функцию
    unsigned long long (*factPtr)(unsigned int) = &factorial;
    std::cout << "Вызов через указатель на функцию 6! = " << factPtr(6)
<< "
";
    return 0;
}
```

Результат выполнения (stdout):

```
После modifyValue: x (по значению) = 5, y (по ссылке) = 105
compute(3, 4) [по умолчанию mult=1] = 7
compute(3, 4, 10) [явно mult=10] = 70
compute(2.5, 4.0) [перегрузка double] = 10
Вызов через указатель на функцию 6! = 720
```

Тонкость / на что обратить внимание: Осторожно с ловушкой перегрузки и параметров по умолчанию: если объявить две функции void f(int a) и void f(int a, int b = 10), то сам факт их объявления не вызовет ошибки, но попытка вызвать f(5) с одним аргументом приведёт к ошибке компиляции (Ambiguous call), так как подойдут обе функции одновременно!

ВОПРОС №5 ИЗ 8 · ТИПЫ ДАННЫХ И ПАМЯТЬ

5. Побитовые операторы, сдвиги, основные приёмы.

Побитовые операции (|, &, ^, ~) и сдвиги (<<, >>), разница между логическим и арифметическим сдвигом вправо, быстрое умножение/деление на 2^n, управление i-м битом числа, вычисление чётности битов (parity) за O(log W) и реализация скалярного произведения векторов над двоичным полем GF(2).

Ключевые тезисы:

- Логический сдвиг вправо (для unsigned типов) заполняет освобождающиеся старшие биты нулями (0); арифметический сдвиг вправо (для signed типов) копирует в старшие биты знаковый бит (сохраняя знак отрицательного числа).
- Операции с i-м битом: установить в 1 → x |= (1u << i); сбросить в 0 → x &= ~(1u << i); инвертировать → x ^= (1u << i); проверить i-й бит → (x >> i) & 1u.
- Чётность битов (parity — сумма всех битов по mod 2) для 32-битного числа вычисляется за log2(32) = 5 шагов с помощью каскада XOR со сдвигами: x ^= x >> 16; x ^= x >> 8; x ^= x >> 4; x ^= x >> 2; x ^= x >> 1; return x & 1;.
- Скалярное произведение двух битовых векторов над полем GF(2) (по модулю 2): побитовое И (a & b) перемножает соответствующие координаты, а бит чётности parity(a & b) складывает эти произведения по модулю 2!

1. Побитовые операторы (|, &, ^, ~) и логический vs арифметический сдвиг

Побитовые операторы применяются к целочисленным типам и выполняют операцию параллельно над каждой парой соответствующих разрядов: 1) & (побитовое И / конъюнкция: 1 & 1 = 1, иначе 0; составной &=); 2) | (побитовое ИЛИ / дизъюнкция: 0 | 0 = 0, иначе 1; составной |=); 3) ^ (побитовое исключающее ИЛИ / XOR / сложение по модулю 2 без переноса: 1 ^ 0 = 1, 0 ^ 1 = 1, а для одинаковых битов 0 ^ 0 = 0 и 1 ^ 1 = 0; составной ^=); 4) ~ (унарная побитовая инверсия / дополнение до 1: меняет все 0 на 1 и 1 на 0; в C++ нет отдельного составного оператора ~=, так как ~ унарный, поэтому пишут x = ~x).

Побитовые сдвиги << (влево, составной <<=) и >> (вправо, составной >>=): при сдвиге влево x << n все биты сдвигаются к старшим разрядам, а справа младшие n

бит всегда заполняются нулями. При сдвиге вправо x >> n различают два вида сдвига:

- Логический сдвиг вправо (Logical Right Shift): применяется к БЕЗЗНАКОВЫМ типам (unsigned int, uint32_t). Освобождающиеся слева старшие разряды всегда заполняются НУЛЯМИ (0).
- Арифметический сдвиг вправо (Arithmetic Right Shift): применяется к ЗНАКОВЫМ типам (int, int32_t, начиная с C++20 закреплён в стандарте). Освобождающиеся слева старшие разряды заполняются копиями ЗНАКОВОГО БИТА (если число было положительным — нулями, если отрицательным — единицами 1), благодаря чему сохраняется знак числа при делении на степень двойки!

2. Быстрое умножение/деление на 2^n и приёмы работы с i-м битом

Сдвиг влево на n позиций эквивалентен умножению числа на 2^n: x << n == x * 2^n. Сдвиг неотрицательного числа вправо на n позиций эквивалентен целочисленному делению на 2^n: x >> n == x / 2^n.

Шпаргалка по манипуляции i-м битом числа x (биты нумеруются с 0 от младшего к старшему):

Задача	**Битовая формула на C++**	**Объяснение работы маски**
Установить i-й бит в 1	x |= (1u << i);	Маска (1u << i) содержит 1 только в i-м разряде; ИЛИ устанавливает этот бит в 1.
Сбросить i-й бит в 0	x &= ~(1u << i);	Инвертированная маска ~(1u << i) содержит 0 только в i-м разряде, а в остальных 1.
Инвертировать i-й бит (0↔1)	x ^= (1u << i);	Свойство XOR: b ^ 1 = ~b (инверсия бита), а b ^ 0 = b (остальные биты не меняются).
Проверить / получить i-й бит	int bit = (x >> i) & 1u;	Сдвигаем i-й бит на нулевую позицию и изолируем его маской & 1.
Сбросить младший единичный бит	x &= (x - 1);	Классический приём Кернигана для быстрого подсчёта единичных битов.

3. Сумма всех битов по mod 2 (Parity) за O(log W) и скалярное произведение над GF(2)

Сумма всех битов числа по модулю 2 (бит чётности — Parity bit) равна 1, если в двоичной записи числа нечётное количество единиц, и 0, если чётное. Так как сложение по модулю 2 — это операция XOR (^), нам нужно вычислить b_31 ^ b_30 ^ ... ^ b_1 ^ b_0.

Как вычислить parity 32-битного числа за O(log2(32)) = 5 операций вместо цикла на 32 итерации? Используем метод «разделяй и властвуй» (параллельное сворачивание половин через XOR): 1) x ^= (x >> 16); — складывает по mod 2 старшие 16 бит с младшими 16 битами; 2) x ^= (x >> 8); — складывает старшие 8 из оставшихся 16 с младшими 8; 3) x ^= (x >> 4);; 4) x ^= (x >> 2);; 5) x ^= (x >> 1);. В результате в самом младшем нулевом бите (x & 1u) оказывается сумма всех 32 исходных битов по модулю 2!

Реализация скалярного произведения векторов над двоичным полем GF(2) (поле Галуа из элементов {0, 1}, где умножение — это логическое И &, а сложение — это сложение по модулю 2 ^): пусть два битовых вектора размерности N упакованы в беззнаковые числа uint32_t u и uint32_t v (или массивы uint64_t для длинных векторов). По формуле скалярного произведения <u, v> = (u_0 & v_0) ^ (u_1 & v_1) ^ ... ^ (u_{n-1} & v_{n-1}). Значит, достаточно: 1) выполнить побитовое И prod = u & v (получим вектор попарных произведений координат за 1 такт CPU); 2) вычислить бит чётности parity(prod) за O(log W)!

Пример кода C++: Операции с i-м битом, Parity за O(log n) и скалярное произведение над полем GF(2)

Демонстрация управления битами, логарифмического вычисления чётности битов и скалярного произведения двух двоичных векторов.

```
#include <iostream>
#include <cstdint>

// Вычисление суммы всех битов по mod 2 (parity) за O(log2(32)) = 5
операций
uint32_t parityLog(uint32_t x) {
    x ^= (x >> 16);
    x ^= (x >> 8);
    x ^= (x >> 4);
    x ^= (x >> 2);
    x ^= (x >> 1);
```

```
    return x & 1u;
}

// Скалярное произведение двух битовых векторов над двоичным полем GF(2)
uint32_t dotProductGF2(uint32_t vecA, uint32_t vecB) {
    // 1. Покоординатное умножение в GF(2) — это побитовое И (&)
    uint32_t bitwiseProd = vecA & vecB;
    // 2. Сумма произведений по mod 2 — это бит чётности (parity)
    return parityLog(bitwiseProd);
}

int main() {
    uint32_t mask = 0b00001010; // 10 (установлены биты 1 и 3)
    mask |= (1u << 2);          // Устанавливаем 2-й бит в 1 ->
0b00001110 (14)
    mask &= ~(1u << 1);         // Сбрасываем 1-й бит в 0    ->
0b00001100 (12)
    mask ^= (1u << 0);          // Инвертируем 0-й бит       ->
0b00001101 (13: три единицы)

    std::cout << "mask = " << mask << ", parityLog(13) = " <<
parityLog(mask) << "
";

    // Вектор A = (1, 1, 0, 1) -> 0b1101, Вектор B = (1, 0, 1, 1) ->
0b1011
    // <A, B> = (1*1) ^ (1*0) ^ (0*1) ^ (1*1) = 1 ^ 0 ^ 0 ^ 1 = 0
    uint32_t vecA = 0b1101;
    uint32_t vecB = 0b1011;
    std::cout << "Скалярное произведение над GF(2) <1101, 1011> = "
              << dotProductGF2(vecA, vecB) << "
";
    return 0;
}
```

Результат выполнения (stdout):

```
mask = 13, parityLog(13) = 1
Скалярное произведение над GF(2) <1101, 1011> = 0
```

Тонкость / на что обратить внимание: Запомните на зачёте: при работе с битовыми масками всегда используйте беззнаковую единицу 1u << i (или 1ULL << i для 64-битных чисел)! Сдвиг знаковой единицы 1 << 31 в 32-битном int приводит к переполнению знакового типа (Undefined Behavior в C++17).

ВОПРОС №6 ИЗ 8 · УКАЗАТЕЛИ, ССЫЛКИ И ПАМЯТЬ

6. Процесс трансляции C++ кода в исполняемый файл. Области памяти исполняемой программы на C++.

Подробный конвейер сборки программы на C++ из 4 стадий (Препроцессор → Компилятор → Ассемблер → Линкер) и архитектура адресного пространства процесса из 5 классических сегментов памяти (.text, .data, .bss, Heap, Stack) с примерами размещения переменных.

Ключевые тезисы:

- 4 этапа трансляции: 1) Препроцессор (.cpp → .i), 2) Компилятор (.i → .s ассемблерный код), 3) Ассемблер (.s → .o / .obj объектный машинный код), 4) Компоновщик / Линкер (.o + библиотеки → исполняемый файл .exe / ELF).
- 5 сегментов памяти программы: Текстовый (.text — машинный код), Сегмент данных (.data — инициализированные глобальные/static), Сегмент .bss (неинициализированные глобальные/static, заполняется нулями), Куча (Heap — new/delete), Стек (Stack — локальные переменные и вызовы функций).
- Стек работает по принципу LIFO (Last In, First Out) за O(1) путём простого сдвига регистра указателя вершины стека (RSP / ESP).

1. Четыре классических этапа трансляции C++ кода в исполняемый файл

Превращение исходного файла main.cpp в готовый к запуску бинарный исполняемый файл проходит через четыре последовательных инструмента цепочки сборки (Toolchain):

Этап и инструмент	Вход → Выход (расширения в GCC)	Что именно происходит на этом этапе
1. Препроцессинг (Препроцессор cpp)	Исходный код (.cpp) → Препроцессированный код (.i / .ii) Команда: g++ -E main.cpp	Текстовая обработка: рекурсивно вставляет содержимое файлов по директивам #include, раскрывает макросы #define, обрабатывает условную компиляцию

Этап и инструмент	Вход → Выход (расширения в GCC)	Что именно происходит на этом этапе
		(#ifdef/#ifndef) и удаляет все комментарии.
2. Компиляция (Компилятор cc1plus)	Препроцессированный код (.i) → Ассемблерный код (.s) Команда: g++ -S main.i	Выполняет лексический, синтаксический (построение AST-дерева) и семантический анализ (проверка типов), инстанцирует шаблоны, оптимизирует код и переводит его в инструкции языка Ассемблера целевого процессора.
3. Ассемблирование (Ассемблер as)	Ассемблерный код (.s) → Объектный файл (.o / .obj) Команда: g++ -c main.s	Переводит текстовые мнемоники ассемблера (mov, add, call) в двоичный машинный код и формирует таблицу символов (экспортируемые и неразрешённые внешние адреса функций).
4. Компоновка / Линковка (Линкер ld / link)	Объектные файлы (.o) + Библиотеки (.a/.so/.lib) → Исполняемый файл (a.out / .exe)	Объединяет секции всех объектных файлов проекта, разрешает перекрёстные ссылки между модулями (подставляет реальные адреса вызываемых функций) и формирует заголовки исполняемого файла ОС (PE на Windows, ELF на Linux).

2. Пять сегментов памяти исполняемой программы и принцип работы Стека

Когда операционная система загружает скомпилированный исполняемый файл в виртуальную память, адресное пространство процесса делится на 5 классических областей (сегментов):

- 1. Текстовый сегмент (Text / Code segment, .text): хранит скомпилированные машинные инструкции функций программы. Имеет права доступа «Только чтение и исполнение» (Read + Execute) для защиты от случайной перезаписи кода данными.
- 2. Сегмент инициализированных данных (Data segment, .data): хранит глобальные и статические (static) переменные, которым программист явно

задал ненулевое начальное значение в коде (например, int globalInit = 42;). Также рядом выделяют секцию .rodata для строковых литералов и глобальных констант.

- 3. Сегмент неинициализированных данных (BSS — Block Started by Symbol, .bss): хранит глобальные и static переменные, которые не были явно инициализированы в коде (или инициализированы нулём). В самом файле на диске они не занимают места — ОС автоматически заполняет весь сегмент .bss нулями при старте программы!
- 4. Куча (Heap / Free store): область динамической памяти, растущая от младших адресов к старшим. Память здесь выделяется вручную оператором new (или malloc) и освобождается через delete (или free). Время жизни объектов контролирует сам программист.
- 5. Стек (Stack): область памяти в верхних адресах (обычно растёт сверху вниз навстречу куче), работающая по принципу магазинной очереди LIFO (Last In, First Out — «последним пришёл, первым ушёл»).

3. Базовые принципы работы Стека (Call Stack) и распределение переменных по 4 сегментам данных

Аппаратный стек процессора управляется двумя специальными регистрами: регистром указателя вершины стека RSP (Stack Pointer) и регистром базы текущего кадра RBP (Base/Frame Pointer). При каждом вызове функции в стеке за константное время O(1) (простым вычитанием размера локальных переменных из регистра RSP) формируется Стековый кадр (Stack Frame), в который помещаются: 1) переданные аргументы функции (если не поместились в регистры), 2) адрес возврата (Return Address — куда вернуться после завершения функции), 3) сохранённый старый RBP и 4) все локальные автоматические переменные функции. При выполнении инструкции return указатель RSP мгновенно сдвигается обратно, и память кадра автоматически считается свободной.

Сводная таблица объявления переменных во всех 4 сегментах данных (помимо текстового сегмента кода .text):

Сегмент памяти	**Как объявить переменную в C++**	**Время жизни и очистка**
1. Стек (Stack)	Локальная переменная внутри функции: int a = 10;	Создаётся при входе в блок { }, удаляется автоматически при выходе.

Сегмент памяти	Как объявить переменную в C++	Время жизни и очистка
2. Куча (Heap)	Динамическое выделение через new: int* p = new int(20);	Живёт в куче от вызова new до явного вызова delete p;.
3. Сегмент .data	Глобальная или static с ненулевой инициализацией: int g = 42;	Хранится в образе файла; живёт всё время работы программы.
4. Сегмент .bss	Глобальная или static неинициализированная: int g_zero;	Зануляется ОС при запуске процесса; живёт всё время работы программы.

Пример кода C++: Размещение переменных во всех 4 сегментах данных (.data, .bss, Stack, Heap) и .text

Наглядный код с объявлением переменных в каждом из сегментов памяти процесса и выводом их адресов.

```
#include <iostream>

// 1. Сегмент .data: глобальная переменная с явной ненулевой
инициализацией
int g_initializedData = 2026;

// 2. Сегмент .bss: глобальная неинициализированная переменная
(автоматически = 0)
int g_uninitializedBss;

// Сама функция main() (её машинный код) находится в текстовом сегменте
(.text)
int main() {
    // Также в .data и .bss попадают локальные static-переменные:
    static int s_inData = 100; // Сегмент .data
    static int s_inBss;        // Сегмент .bss (обнуляется при старте)

    // 3. Стек (Stack): локальная автоматическая переменная
    int localOnStack = 77;

    // 4. Куча (Heap): динамически выделенная переменная через оператор
new
    int* dynamicOnHeap = new int(999);

    std::cout << "1. Сегмент .data (g_initializedData): " <<
g_initializedData
              << " [адрес: " << &g_initializedData << "]
";
    std::cout << "2. Сегмент .bss  (g_uninitializedBss): " <<
```

```
g_uninitializedBss
              << "    [адрес: " << &g_uninitializedBss << "]
";
    std::cout << "3. Стек (Stack)  (localOnStack):       " <<
localOnStack
              << "   [адрес: " << &localOnStack << "]
";
    std::cout << "4. Куча (Heap)   (*dynamicOnHeap):     " <<
*dynamicOnHeap
              << "  [адрес: " << dynamicOnHeap << "]
";

    delete dynamicOnHeap; // Освобождаем память в куче
    return 0;
}
```

Результат выполнения (stdout):

```
1. Сегмент .data (g_initializedData): 2026 [адрес: 0x55a1c0104010]
2. Сегмент .bss  (g_uninitializedBss): 0    [адрес: 0x55a1c0104018]
3. Стек (Stack)  (localOnStack):       77   [адрес: 0x7ffd8b4e2a1c]
4. Куча (Heap)   (*dynamicOnHeap):     999  [адрес: 0x55a1c193deb0]
```

Тонкость / на что обратить внимание: Частый вопрос на зачёте: где хранится сам указатель int* dynamicOnHeap = new int(999);, а где хранится число 999? Ответ: сама 8-байтная переменная-указатель dynamicOnHeap объявлена внутри main(), поэтому лежит на СТЕКЕ (Stack), а вот 4 байта с числом 999, на которые она указывает, выделены в КУЧЕ (Heap)!

ВОПРОС №7 ИЗ 8 · ФУНКЦИИ И АЛГОРИТМЫ

7. Алгоритмы. Пос-ть Фиббоначи. Линейный и бинарный поиск. Reverse, shift массива. Соответсвующие оценки сложности. Swap целых чисел через + и xor. Вычисление бита чётности (parity bit) за n и log(n).

Оценка асимптотической сложности О-большое, рекурсивное O(2^n) и итеративное O(n) вычисление чисел Фибоначчи, императивный и декларативный стили, линейный поиск (прямой, с конца, двусторонний) и бинарный поиск O(log

n), разворот массива (reverse) на месте и циклический сдвиг (shift) за O(n) через 3 разворота, обмен переменных без третьей переменной (через +/- и через XOR) и вычисление бита чётности за O(n) и O(log n).

Ключевые тезисы:

- Сложность Фибоначчи: наивная рекурсия — O(2^n) по времени и O(n) по памяти стека; циклический алгоритм — O(n) по времени и O(1) по памяти.
- Разворот массива (reverse) на месте выполняется за O(n) времени и O(1) памяти; циклический сдвиг массива на k позиций за O(n) времени и O(1) памяти реализуется через 3 вызова reverse!
- Swap без временной переменной: арифметический (a = a + b; b = a - b; a = a - b;) и побитовый (a ^= b; b ^= a; a ^= b; — требует проверки &a != &b, иначе обнулит переменную!).
- Бит чётности (parity bit) вычисляется за O(n) простым циклом по битам (или за число единичных битов через x &= x - 1) и за O(log n) путём каскадного XOR половин битового слова.

1. Понятие сложности алгоритма (Big O) и числа Фибоначчи (рекурсия vs цикл, императивный vs декларативный стиль)

Асимптотическая сложность алгоритма O(f(n)) характеризует скорость роста числа элементарных операций (временная сложность — Time Complexity) или объёма дополнительной памяти (пространственная сложность — Space Complexity) при стремлении размера входных данных n к бесконечности, отбрасывая константные множители и младшие слагаемые.

Последовательность Фибоначчи задаётся рекуррентно: F(0) = 0, F(1) = 1, F(n) = F(n-1) + F(n-2) для n >= 2 (ряд: 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55...).

- Рекурсивная реализация (декларативный стиль — описывает математическое определение «ЧТО есть F(n)»): return (n <= 1) ? n : fib(n-1) + fib(n-2);. Имеет катастрофическую экспоненциальную сложность по времени O(2^n) (точнее O(phi^n), где phi ≈ 1.618), так как одни и те же значения F(k) пересчитываются заново миллионы раз, и пространственную сложность O(n) из-за глубины стека вызовов.

- Циклическая реализация (императивный стиль — пошагово описывает «КАК изменять состояние переменных в памяти»): хранит только два предыдущих числа prev и curr в цикле от 2 до n. Имеет линейную сложность по времени O(n) и константную по памяти O(1).

2. Линейный поиск (с разных сторон), бинарный поиск, Reverse и циклический Shift через 3 Reverse

Линейный поиск в неотсортированном массиве проверяет элементы подряд: 1) Слева направо (от 0 до n-1) — находит первое вхождение; 2) Справа налево (от n-1 до 0) — находит последнее вхождение; 3) Двусторонний поиск (навстречу с двух концов i = 0, j = n - 1 одновременно) — делает в 2 раза меньше итераций цикла, но асимптотика во всех случаях линейная: O(n) по времени и O(1) по памяти.

Бинарный поиск работает ТОЛЬКО на отсортированном массиве: на каждом шаге сравнивает ключ с серединным элементом mid = left + (right - left) / 2 и отбрасывает половину массива. Временная сложность — O(log2 n), пространственная — O(1) (для итеративной версии).

Разворот массива «на месте» (In-place Reverse): два индекса l = 0 и r = n - 1 идут навстречу друг другу (while (l < r)), меняя местами элементы swap(a[l++], a[r--]). Выполняет ровно n/2 обменов: время O(n), доп. память O(1).

Элегантный алгоритм циклического сдвига массива (Shift / Rotate) на k позиций за O(n) времени и O(1) памяти через 3 разворота (3 reverses): Пусть нужно циклически сдвинуть массив влево на k = k % n позиций (разделив массив на префикс A длины k и суффикс B длины n - k, чтобы из AB получить BA): 1) Разворачиваем префикс A: reverse(0, k - 1) → получаем A^R B; 2) Разворачиваем суффикс B: reverse(k, n - 1) → получаем A^R B^R; 3) Разворачиваем весь массив целиком: reverse(0, n - 1) → так как (A^R B^R)^R = BA, получаем сдвинутый массив! (Для сдвига вправо на k достаточно сделать сдвиг влево на n - (k % n), либо сначала развернуть весь массив, затем первые k и последние n - k элементов).

3. Swap целых чисел через + и XOR. Вычисление бита чётности (Parity bit) за O(n) и O(log n)

Как поменять местами две целочисленные переменные a и b без использования третьей временной переменной tmp (O(1) памяти, 0 дополнительных регистров)?

- Вычисление бита чётности (Parity bit) за O(n), где n — разрядность числа в битах (например, 32): в цикле while (x != 0) прибавляем по модулю 2 младший бит: parity ^= (x & 1u); x >>= 1;. (Ускоренная версия Кернигана за число единичных битов k: while (x) { parity ^= 1u; x &= (x - 1); }).
- Вычисление бита чётности (Parity bit) за O(log n): последовательное сворачивание половин слова через XOR со сдвигом на n/2, n/4, ..., 1: для 32 бит x ^= x >> 16; x ^= x >> 8; x ^= x >> 4; x ^= x >> 2; x ^= x >> 1; return x & 1u; — ровно log2(32) = 5 шагов!

Способ обмена Swap	Три шага алгоритма	Особенности и подводные камни
Через сложение и вычитание (+ / -)	1) a = a + b; 2) b = a - b; // (a+b)-b = a 3) a = a - b; // (a+b)-a = b	Риск переполнения знакового типа int при больших числах a + b, но в беззнаковой арифметике работает корректно даже при переполнении по модулю 2^W.
Через побитовый XOR (^)	1) a = a ^ b; 2) b = a ^ b; // (a^b)^b = a 3) a = a ^ b; // (a^b)^a = b	Никогда не вызывает арифметического переполнения! ОПАСНОСТЬ: если передать одну и ту же ячейку памяти swapXor(x, x) (&a == &b), на первом же шаге x ^ x даст 0 и сотрёт число! Поэтому нужна проверка if (&a != &b).

Пример кода C++: Reverse на месте, циклический Shift через 3 Reverse, Swap через + и XOR, Parity за n и log(n)

Полная реализация всех алгоритмов билета №7 с проверкой на тестовом массиве.

```
#include <iostream>
#include <cstdint>

// 1. Обмен через + и - и через XOR (с защитой от совпадения адресов &a
== &b)
void swapAddSub(int& a, int& b) {
    if (&a == &b) return;
    a = a + b;
    b = a - b;
    a = a - b;
}

void swapXor(int& a, int& b) {
    if (&a == &b) return; // Иначе a ^= a обнулит переменную!
    a ^= b;
    b ^= a;
```

```
    a ^= b;
}

// 2. Разворот участка массива [left..right] "на месте" за O(n) времени и
O(1) памяти
void reverseRange(int arr[], int left, int right) {
    while (left < right) {
        swapXor(arr[left], arr[right]);
        ++left;
        --right;
    }
}

// 3. Циклический сдвиг влево на k позиций через 3 разворота за O(n)
времени и O(1) памяти
void shiftLeftByThreeReverses(int arr[], int n, int k) {
    if (n <= 1) return;
    k %= n;
    if (k == 0) return;
    reverseRange(arr, 0, k - 1);     // 1. Разворачиваем первые k
элементов
    reverseRange(arr, k, n - 1);     // 2. Разворачиваем оставшиеся n - k
элементов
    reverseRange(arr, 0, n - 1);     // 3. Разворачиваем весь массив
целиком
}

// 4. Вычисление бита чётности (Parity) за O(n) и за O(log n)
uint32_t parityLinear(uint32_t x) {
    uint32_t p = 0;
    while (x != 0) {
        p ^= (x & 1u);
        x >>= 1;
    }
    return p;
}

uint32_t parityLogN(uint32_t x) {
    x ^= (x >> 16);
    x ^= (x >> 8);
    x ^= (x >> 4);
    x ^= (x >> 2);
    x ^= (x >> 1);
    return x & 1u;
}

int main() {
    int a = 15, b = 99;
    swapAddSub(a, b);
    std::cout << "После swapAddSub: a = " << a << ", b = " << b << "
";
```

```
    int arr[6] = {1, 2, 3, 4, 5, 6};
    shiftLeftByThreeReverses(arr, 6, 2); // Сдвиг влево на 2 -> {3, 4, 5,
6, 1, 2}
    std::cout << "Массив после циклического сдвига влево на 2: ";
    for (int x : arr) std::cout << x << " ";
    std::cout << "
";

    uint32_t val = 0b10110100; // Четыре единицы -> чётность равна 0
    std::cout << "Parity(0b10110100): O(n) = " << parityLinear(val)
              << ", O(log n) = " << parityLogN(val) << "
";
    return 0;
}
```

Результат выполнения (stdout):

```
После swapAddSub: a = 99, b = 15
Массив после циклического сдвига влево на 2: 3 4 5 6 1 2
Parity(0b10110100): O(n) = 0, O(log n) = 0
```

Тонкость / на что обратить внимание: Обязательно запомните вопрос с подвохом на зачёте: почему swapXor(arr[i], arr[j]) в сортировке может внезапно занулить элемент массива? Потому что при i == j оба параметра ссылаются на одну и ту же ячейку памяти (&a == &b), и первая же строчка a ^= b вычисляет x ^ x = 0, уничтожая значение!

ВОПРОС №8 ИЗ 8 · ФУНКЦИИ И АЛГОРИТМЫ

8. Алгоритмы. Получение случайного числа в заданном диапазоне при помощи ф-ции rand(), принцип её работы. Сортировки n^2: выбором, вставкой, пузырьком (оптимизации: перемешиванием, расчёской). Сортировки n^log(n): слиянием, быстрая, пирамидальная. Сортировки n: подсчётом.

Исчерпывающее руководство по генерации псевдослучайных чисел (rand, srand, time, формула диапазона [min, max]) и полной классификации алгоритмов

сортировки: понятия устойчивости (stability), работы «на месте» (in-place), квадратичные сортировки O(n^2) с оптимизациями (шейкерная, расчёской), быстрые сортировки O(n log n) (Merge, Quick, Heap) и линейная сортировка подсчётом O(n + k).

Ключевые тезисы:

- Генератор rand() работает по принципу линейного конгруэнтного метода: X_{n+1} = (a * X_n + c) mod m. Для инициализации зерна (seed) текущим временем один раз в начале main() вызывают srand(time(nullptr)). Формула диапазона [min, max]: min + rand() % (max - min + 1).
- Сортировка называется «на месте» (in-place), если она использует O(1) (или O(log n) для стека) дополнительной памяти; устойчивой (stable) — если она сохраняет взаимный порядок элементов с равными ключами.
- Любая сортировка, основанная исключительно на попарном сравнении элементов (Comparison sort), не может работать быстрее нижней оценки O(n log n) в худшем/среднем случае; сортировка подсчётом (Counting sort) обходит этот барьер за счёт прямой адресации в массиве частот за O(n + k).

1. Принцип работы ГПСЧ rand(), srand(), time() и формула диапазона [min, max]

Функция rand() из заголовка <cstdlib> возвращает псевдослучайное целое число в диапазоне от 0 до константы RAND_MAX (обычно 32767 или 2147483647). Почему числа называются ПСЕВДОслучайными? Потому что внутри rand() хранится скрытая статическая переменная текущего состояния (seed / зерно), а каждое следующее число вычисляется по детерминированной математической формуле Линейного Конгруэнтного Генератора (LCG): X_{k+1} = (A * X_k + C) % M.

Если не вызвать srand(), начальное зерно всегда равно 1, и при каждом перезапуске программа будет выдавать одну и ту же последовательность чисел! Чтобы последовательность каждый раз была новой, РОВНО ОДИН РАЗ в начале main() вызывают функцию установки зерна srand(static_cast<unsigned>(time(nullptr))), где time(nullptr) из <ctime> возвращает текущее системное время в секундах с 1 января 1970 года (Unix Epoch).

Формула получения случайного целого числа в произвольном включительном отрезке [min, max]: int r = min + rand() % (max - min + 1); (остаток от деления на длину интервала (max - min + 1) даёт смещение от 0 до max - min, к которому прибавляется левая граница min).

2. Классификация сортировок и алгоритмы за O(n^2) с оптимизациями

Две фундаментальные характеристики любого алгоритма сортировки: 1) Сортировка «на месте» (In-place sort) — переставляет элементы прямо внутри исходного массива, требуя лишь O(1) дополнительной памяти (или O(log n) на стек рекурсии в QuickSort); 2) Устойчивость (Stability) — устойчивая сортировка гарантированно НЕ меняет относительный порядок двух элементов с одинаковыми ключами (это критично при сортировке таблиц по нескольким столбцам).

Квадратичные сортировки сравнением O(n^2) и их оптимизации:

- 1. Сортировка выбором (Selection Sort): на каждом шаге i от 0 до n-2 находит индекс минимального элемента в неотсортированном хвосте [i..n-1] и меняет его местами с arr[i]. Всегда выполняет O(n^2) сравнений (даже на уже отсортированном массиве!), но делает минимум обменов — не более n - 1. Работает на месте O(1), НЕустойчивая.
- 2. Сортировка вставками (Insertion Sort): перебирает элементы с i = 1 до n-1 и вставляет текущий элемент key = arr[i] на нужное место в уже отсортированную левую часть [0..i-1], сдвигая большие элементы вправо. В худшем и среднем случае — O(n^2), но на уже отсортированном (или почти отсортированном) массиве работает за линейное время O(n)! Работает на месте O(1), УСТОЙЧИВАЯ.
- 3. Сортировка пузырьком (Bubble Sort): многократно проходит по массиву, меняя местами соседние элементы arr[j] > arr[j+1]. За один проход самый большой элемент «всплывает» в конец. Базовая сложность O(n^2), с флагом отсутствия обменов за проход — O(n) в лучшем случае. На месте O(1), УСТОЙЧИВАЯ.
- 4. Оптимизация пузырька №1 — Перемешиванием / Шейкерная сортировка (Cocktail Shaker Sort): решает проблему «черепах» (маленьких элементов в самом конце массива, которые в обычном пузырьке сдвигаются влево всего на 1 позицию за проход). Чередует проходы: сначала слева направо (всплывает максимум вправо), затем сразу справа налево (тонет минимум влево), сужая

границы [left, right]. Сложность остаётся O(n^2), но на практике быстрее пузырька. Устойчивая, на месте O(1).

- 5. Оптимизация пузырька №2 — Сортировка расчёской (Comb Sort): полностью уничтожает «черепах» за счёт того, что сначала сравнивает и меняет элементы не на расстоянии 1, а на большом шаге gap = n / 1.247. На каждой итерации шаг делится на фактор уменьшения k ≈ 1.247 (1.3), пока gap не дойдёт до 1, где алгоритм завершается как обычный пузырёк. В среднем работает почти за O(n log n) (худшая O(n^2)), на месте O(1), НЕустойчивая.

3. Сортировки за O(n log n) (Merge, Quick, Heap) и линейная сортировка подсчётом O(n)

В теории алгоритмов доказано, что любая сортировка, основанная только на сравнениях элементов (Comparison sort), в худшем и среднем случае требует не менее log2(n!) = O(n log n) сравнений:

Алгоритм сортировки	**Принцип работы**	**Время (Лучшее / Среднее / Худшее)**	**Доп. память и Устойчивость**
Слиянием (Merge Sort)	Разделяй и властвуй: массив делится пополам, обе половины рекурсивно сортируются и сливаются (merge) в один упорядоченный массив двумя указателями.	O(n log n) / O(n log n) / O(n log n)	Память O(n) (НЕ на месте!), УСТОЙЧИВАЯ.
Быстрая сортировка Хоара (QuickSort)	Выбирается опорный элемент (pivot). Разбиение (partition) перекидывает элементы < pivot влево, а > pivot вправо, после чего обе части сортируются рекурсивно.	O(n log n) / O(n log n) / O(n^2) (при неудачном pivot)	Память O(log n) на стек (на месте), НЕустойчивая. Самая быстрая на практике!
Пирамидальная / Кучей (Heap Sort)	Сначала массив за O(n) перестраивается в бинарную max-кучу (Binary Heap, где родитель >= детей). Затем максимум (корень arr[0]) меняется с	O(n log n) / O(n log n) / O(n log n)	Память строго O(1) (на месте!), НЕустойчивая.

Алгоритм сортировки	Принцип работы	Время (Лучшее / Среднее / Худшее)	Доп. память и Устойчивость
	концом массива, и куча восстанавливается (siftDown) за O(log n).		
Подсчётом (Counting Sort — non-comparison)	Не сравнивает элементы! Создаёт массив частот count[0..K], подсчитывает сколько раз встретилось каждое число, и выписывает их по порядку.	O(n + K) во всех случаях (где K — диапазон значений)	Память O(K) (или O(n + K)), УСТОЙЧИВАЯ (при сборке через префиксные суммы). Применима только к целым числам из небольшого диапазона!

Пример кода C++: ГПСЧ rand() в диапазоне [min, max], сортировка расчёской, быстрая сортировка (QuickSort) и сортировка подсчётом

Рабочая демонстрация генерации массива случайных чисел и его сортировки разными классами алгоритмов.

```
#include <iostream>
#include <vector>
#include <cstdlib>
#include <ctime>
#include <algorithm>

// 1. Генерация случайного числа в отрезке [minVal, maxVal]
int randomInRange(int minVal, int maxVal) {
    return minVal + std::rand() % (maxVal - minVal + 1);
}

// 2. Сортировка расчёской (Comb Sort — оптимизация пузырька с шагом gap
/ 1.247)
void combSort(std::vector<int>& a) {
    int n = static_cast<int>(a.size());
    int gap = n;
    bool swapped = true;
    while (gap > 1 || swapped) {
        gap = static_cast<int>(gap / 1.24733);
        if (gap < 1) gap = 1;
        swapped = false;
        for (int i = 0; i + gap < n; ++i) {
```

```
            if (a[i] > a[i + gap]) {
                std::swap(a[i], a[i + gap]);
                swapped = true;
            }
        }
    }
}

// 3. Быстрая сортировка Хоара (QuickSort) за O(n log n)
void quickSort(std::vector<int>& a, int left, int right) {
    if (left >= right) return;
    int i = left, j = right;
    int pivot = a[left + (right - left) / 2]; // Опорный элемент по
центру
    while (i <= j) {
        while (a[i] < pivot) ++i;
        while (a[j] > pivot) --j;
        if (i <= j) {
            std::swap(a[i], a[j]);
            ++i;
            --j;
        }
    }
    if (left < j) quickSort(a, left, j);
    if (i < right) quickSort(a, i, right);
}

// 4. Линейная сортировка подсчётом (Counting Sort) за O(n + K) для чисел
[0..maxVal]
void countingSort(std::vector<int>& a, int maxVal) {
    std::vector<int> count(maxVal + 1, 0);
    for (int x : a) ++count[x]; // Подсчитываем частоту каждого числа за
O(n)
    int pos = 0;
    for (int val = 0; val <= maxVal; ++val) {
        while (count[val]-- > 0) {
            a[pos++] = val;     // Восстанавливаем отсортированный массив
        }
    }
}

int main() {
    std::srand(12345u); // Фиксируем зерно (на практике:
std::srand(std::time(nullptr)))
    std::vector<int> arr(8);
    for (int& x : arr) x = randomInRange(1, 20);

    std::vector<int> a1 = arr, a2 = arr, a3 = arr;
    combSort(a1);
    quickSort(a2, 0, static_cast<int>(a2.size()) - 1);
    countingSort(a3, 20);
```

```
    std::cout << "Исходный массив [1..20]: ";
    for (int x : arr) std::cout << x << " ";
    std::cout << "
После QuickSort / CountingSort: ";
    for (int x : a2) std::cout << x << " ";
    std::cout << "
";
    return 0;
}
```

Результат выполнения (stdout):

```
Исходный массив [1..20]: 18 11 9 5 11 2 4 19
После QuickSort / CountingSort: 2 4 5 9 11 11 18 19
```

Тонкость / на что обратить внимание: На зачёте обязательно помните: если вызывать srand(time(nullptr)) ВНУТРИ цикла или внутри функции randomInRange() при каждом запросе числа, то в течение одной и той же секунды time(nullptr) будет возвращать одинаковое число, генератор будет сбрасываться в одно и то же начальное состояние, и rand() выдаст тысячи одинаковых чисел подряд! Поэтому srand() вызывают строго 1 раз в начале main().